

✓ 1.Linear Regression

1.The reason the Ordinary Least Squares (OLS) fitted line is not well aligned with the majority of the data points is due to the **presence of outliers** in the dataset. OLS minimizes the sum of squared residuals, which means that it heavily penalizes large errors (due to the squaring). As a result, even a few extreme outlier points such as those visibly far from the general linear trend in both the upper right and lower left corners of the plot can disproportionately influence the regression line. These outliers exert a strong “pull” on the fitted line in order to reduce their large squared errors, even if doing so increases the error for the bulk of the data points. In this case, although the majority of the data lies near a horizontal trend with small y values, the OLS line is skewed upwards to partially accommodate the large magnitude outliers.

2.A better fitted line for the inliers is expected under **Scheme 1**, where $a_i = 0.01$ for outliers and $a_i = 1$ for inliers. This is because Scheme 1 **down-weights the contribution of outliers** in the loss function, effectively reducing their influence on the regression. By assigning a very small weight (0.01) to the squared error from outliers, their large errors contribute negligibly to the total loss. In contrast, inliers retain full weight, so the regression line will be optimized to minimize error primarily for the inliers. This aligns the model more closely with the majority of data points, which follow a consistent linear trend, resulting in a line that better fits the inlier distribution.

On the other hand, Scheme 2 does the opposite: it gives outliers five times more weight than inliers. This increases their influence even more than in standard OLS (which treats all points equally), leading to a regression line that is further skewed away from the inliers in an attempt to minimize the weighted loss from the outliers. Therefore, Scheme 2 would worsen the fit for inliers compared to OLS.

Thus, Scheme 1 is the correct choice for obtaining a regression line that better represents the majority (inlier) trend in the data, as it robustly suppresses the impact of extreme outlier values.

3.The reasons why linear regression is not suitable for brain image analysis:

1. fMRI data contains thousands of features but few samples leading to overfitting in linear models.
 2. Brain activity patterns are often nonlinear, which linear regressions cannot capture effectively
 3. Features within and across brain regions are highly correlated violating the independence assumption in linear regression.
 4. Noise in high dimensional data can make the linear model unstable and difficult to interpret reliably.
-

4,5.In the context of brain imaging, each brain region consists of many voxels that are naturally grouped together. Standard LASSO selects individual voxels independently, without considering which region they belong to. This can lead to a situation where only a few scattered voxels are selected from many different regions, making it difficult to interpret which entire brain region is actually important for the cognitive task.

In contrast, Group LASSO is specifically designed to handle this kind of grouped structure. It selects or discards entire groups of voxels (brain regions) together. This makes it much more suitable for identifying which brain regions are most predictive, because it respects the anatomical organization of the brain. It improves interpretability and reduces the risk of selecting noisy individual voxels that don't reflect the behavior of the whole region.

Therefore most suitable method : Group LASSO (Method B)

✓ 2.Logistic Regression

1.Loading data

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load the penguins dataset
df = sns.load_dataset("penguins")
df.dropna(inplace=True)
```

```
# Filter rows for 'Adelie' and 'Chinstrap' classes
selected_classes = ['Adelie', 'Chinstrap']

# Make a copy to avoid warning
df_filtered = df[df['species'].isin(selected_classes)].copy()

# Initialize the LabelEncoder
le = LabelEncoder()

# Encode the species column
y_encoded = le.fit_transform(df_filtered['species'])
df_filtered['class_encoded'] = y_encoded

# Display the filtered and encoded DataFrame
print(df_filtered[['species', 'class_encoded']])

# Split the data into features (X) and target variable (y)
y = df_filtered['class_encoded'] # Target variable
X = df_filtered.drop(['class_encoded'], axis=1)
```

```
↗
   species  class_encoded
0    Adelie              0
1    Adelie              0
2    Adelie              0
4    Adelie              0
5    Adelie              0
..      ...            ...
215  Chinstrap            1
216  Chinstrap            1
217  Chinstrap            1
218  Chinstrap            1
219  Chinstrap            1
```

[214 rows x 2 columns]

2.Training a logistic regression

```
#Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the logistic regression model. Here we are using saga solver to learn weights.
logreg = LogisticRegression(solver='saga')
logreg.fit(X_train, y_train)

# Predict on the testing data
y_pred = logreg.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(logreg.coef_, logreg.intercept_)
```

```
↗ -----
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-1532930499.py in <cell line: 0>()
      4 # Train the logistic regression model. Here we are using saga solver to learn weights.
      5 logreg = LogisticRegression(solver='saga')
----> 6 logreg.fit(X_train, y_train)
      7
      8 # Predict on the testing data

-----
      6 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py in __array__(self, dtype, copy)
    2151 ) -> np.ndarray:
    2152     values = self._values
-> 2153     arr = np.asarray(values, dtype=dtype)
    2154     if (
    2155         astype_is_view(values.dtype, arr.dtype)

ValueError: could not convert string to float: 'Adelie'
```

Next steps: [Explain error](#)

Reason for error:The above error is telling that Logistic Regression tried to process as a string as a numerical input and it failed because Scikit-learn models like LogisticRegression can only work with numbers, not text.

How to fix the error:

1. Use LabelEncoder on y_train if it's a string label like 'Adelie'
2. Use OneHotEncoder or pd.get_dummies() on X_train if it contains string features.

```
from sklearn.preprocessing import OneHotEncoder

# 1. Load and clean the dataset
df = sns.load_dataset("penguins")
df.dropna(inplace=True)

# 2. Filter for Adelie and Chinstrap
selected_classes = ['Adelie', 'Chinstrap']
df_filtered = df[df['species'].isin(selected_classes)].copy()

# 3. Encode the target variable
le=LabelEncoder()
df_filtered['class_encoded']=le.fit_transform(df_filtered['species'])

# 4.Drop target and non_numeric coulms from X
X=df_filtered.drop(['species','class_encoded'],axis=1)

# 5. One-hot encode categorical features in X (like 'island' and 'sex')
X = pd.get_dummies(X, drop_first=True)

# 6. Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Train logistic regression with saga solver
logreg = LogisticRegression(solver='saga')
logreg.fit(X_train, y_train)

# 8. Evaluate
y_pred = logreg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print(logreg.coef_, logreg.intercept_)

➦ Accuracy: 0.58
[[ 2.76007502e-03 -8.11804214e-05  4.63704334e-04 -2.86668107e-04
  1.84431131e-04 -1.04397649e-04  1.11229233e-05] [-8.5000256e-06]
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was reached which means the c
warnings.warn()
```

3.Why does the saga solver perform poorly ?

1. It is designed for large scale and sparse datasets and is specially effective when using L1 or elastic net regularization. If dataset is small saga may be slower and less efficient.
2. If the maximum number of iterations is set too low, it may not converge resulting in suboptimal coefficients and warnings.
3. Poor feature scaling can also hurt performance as saga relies on gradient updates that become unstable if features vary widely in scale.

4.Changing the solver to 'liblinear'

```
# 1. Load and clean the dataset
df = sns.load_dataset("penguins")
df.dropna(inplace=True)

# 2. Filter for Adelie and Chinstrap
selected_classes = ['Adelie', 'Chinstrap']
df_filtered = df[df['species'].isin(selected_classes)].copy()

# 3. Encode the target variable
le=LabelEncoder()
df_filtered['class_encoded']=le.fit_transform(df_filtered['species'])

# 4.Drop target and non_numeric coulms from X
X=df_filtered.drop(['species','class_encoded'],axis=1)
```

```
# 5. One-hot encode categorical features in X (like 'island' and 'sex')
X = pd.get_dummies(X, drop_first=True)

# 6. Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Train logistic regression with saga solver
logreg = LogisticRegression(solver='liblinear')
logreg.fit(X_train, y_train)

# 8. Evaluate
y_pred = logreg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print(logreg.coef_, logreg.intercept_)

➦ Accuracy: 1.00
[[ 1.51524391 -1.39159923 -0.14412226 -0.00365548  0.73456625 -0.56189725
 -0.22642575] [-0.07740379]]
```

5. Reason why 'liblinear' outperformed 'saga':

1. 'liblinear' is a deterministic and exact solver based on coordinate descent therefore it often converges faster and more reliably on small or moderately sized datasets specially when the data is dense and well conditioned.
2. On the other hand 'saga' is a stochastic optimizer that approximates gradients and can be sensitive to insufficient iterations, unscaled features or noisy gradients leading to suboptimal convergence and lower accuracy.

6. Why the model's accuracy (with saga solver) varies with different random state values:

saga is a stochastic (randomized) optimization algorithm. It updates model weights using small, randomly selected batches of data, so the path it takes to reach a solution can differ slightly each time. The random_state controls this randomness. Therefore changing the random state leads to different initial conditions or update sequences which results in variations of accuracy.

This randomness has more impact when the dataset is small, noisy or the number of training iterations is low.

7. Using feature scaling along with 'saga' and 'liblinear'

```
from sklearn.preprocessing import StandardScaler

# Load and clean dataset
df = sns.load_dataset("penguins")
df.dropna(inplace=True)

# Filter for 'Adelie' and 'Chinstrap' only
df = df[df['species'].isin(['Adelie', 'Chinstrap'])].copy()

# Encode target labels
le = LabelEncoder()
df['class_encoded'] = le.fit_transform(df['species']) # Adelie = 0, Chinstrap = 1

# Drop non-numeric and target columns from features
X = df.drop(columns=['species', 'class_encoded'])
X = pd.get_dummies(X, drop_first=True) # One-hot encode 'island' and 'sex'
y = df['class_encoded']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42, test_size=0.2)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Using 'saga':

```
# Logistic Regression with saga
logreg_saga = LogisticRegression(solver='saga', max_iter=1000, random_state=0)
logreg_saga.fit(X_train_scaled, y_train)
y_pred_saga = logreg_saga.predict(X_test_scaled)
```

```
# Accuracy
acc_saga = accuracy_score(y_test, y_pred_saga)
print(f"SAGA solver accuracy: {acc_saga:.2f}")
```

```
↔ SAGA solver accuracy: 1.00
```

Using 'liblinear':

```
# Logistic Regression with liblinear
logreg_liblinear = LogisticRegression(solver='liblinear', max_iter=1000, random_state=0)
logreg_liblinear.fit(X_train_scaled, y_train)
y_pred_liblinear = logreg_liblinear.predict(X_test_scaled)
```

```
# Accuracy
acc_liblinear = accuracy_score(y_test, y_pred_liblinear)
print(f"Liblinear solver accuracy: {acc_liblinear:.2f}")
```

```
↔ Liblinear solver accuracy: 1.00
```

Now the accuracy when using both 'saga' and 'liblinear' is 1.00 . The possible reasoning is as follows:

Scaling allowed the model to optimize effectively by placing all features on a comparable numerical scale. In logistic regression, unscaled features with different magnitudes can distort the learning process specially for resolvers like 'saga' that rely on gradient based optimization causing the model to focus disproportionately on features with larger values.

8. This approach is *not correct* because applying label encoding followed by feature scaling to a categorical feature *introduces misleading numerical relationships*.

Label encoding converts categories like 'red', 'blue', and 'green' into arbitrary integers (e.g., 0, 1, 2), which creates a false sense of order and magnitude among the categories. When a scaler is then applied, it treats these encoded values as continuous, implying mathematical relationships that do not exist in the original data. This can mislead the model into thinking, for instance, that 'green' is greater than 'blue' or that the difference between 'blue' and 'red' is meaningful, which is not the case for nominal categories.

The correct approach is to use one-hot encoding, which creates separate binary columns for each category and preserves the categorical nature of the data without imposing any artificial order or scale. This ensures that the model treats each category independently and avoids introducing bias through improper numerical transformations.

✓ 3.Logistic regression First/Second - Order Methods

1.Generating data

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
```

```
# Generate synthetic data
np.random.seed(0)
centers=[[-5,0],[5,1.5]]
```

```
X_orig,y=make_blobs(n_samples=2000,centers=centers,random_state=5)
transformation=[[0.5,0.5],[-0.5,1.5]]
X_transformed=np.dot(X,transformation)
```

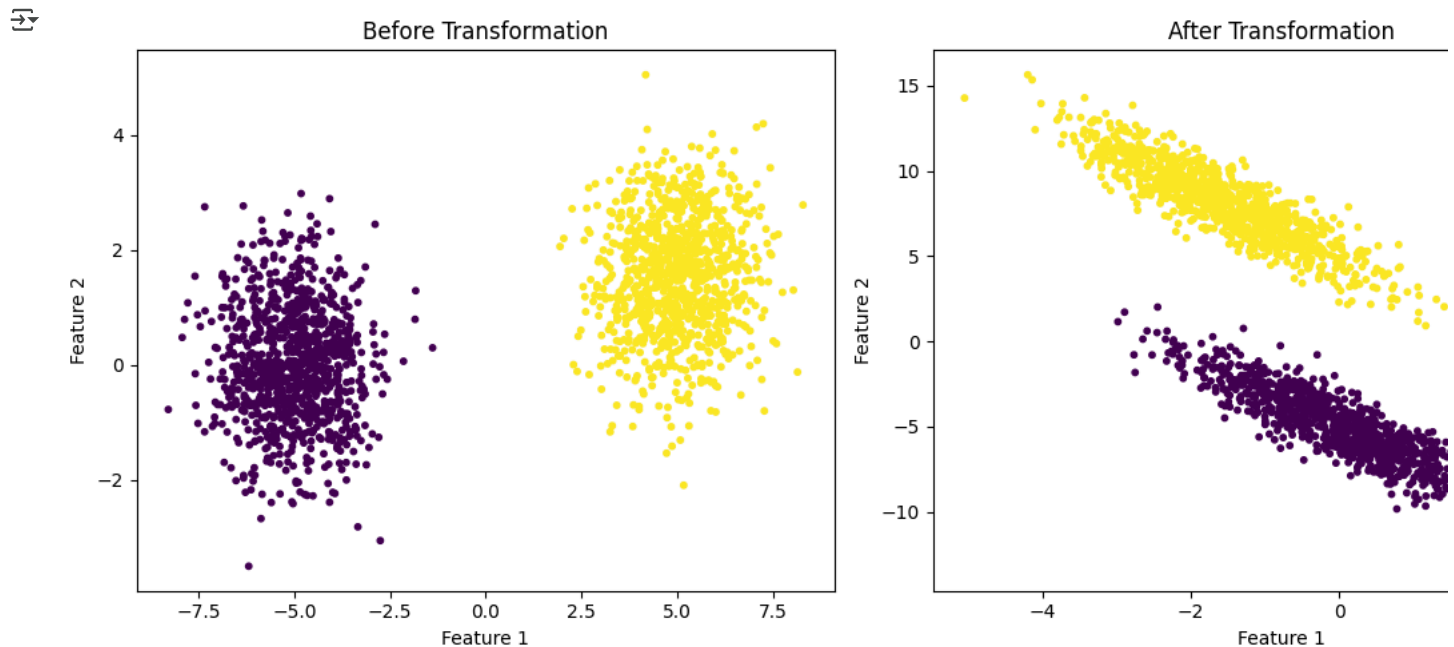
```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.scatter(X_orig[:, 0], X_orig[:, 1], c=y, cmap='viridis', s=10)
plt.title("Before Transformation")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
```

```
plt.subplot(1, 2, 2)
plt.scatter(X_transformed[:, 0], X_transformed[:, 1], c=y, cmap='viridis', s=10)
plt.title("After Transformation")
```

```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
```

```
plt.tight_layout()
plt.show()
```



2. Implementing Batch Gradient Descent

```
scaler=StandardScaler()
X_transformed=scaler.fit_transform(X_transformed)

X=np.hstack((np.ones((X.shape[0],1)),X))

weights=np.zeros(X.shape[1])

def sigmoid(z):
    return 1/(1+np.exp(-z))

alpha=0.1
n_iter=20
losses_gd=[]

for i in range(n_iter):
    z=np.dot(X,weights)
    predictions=sigmoid(z)
    errors=predictions-y
    gradient=np.dot(X.T,errors)/X.shape[0]
    weights -= alpha*gradient

    loss=-np.mean(y*np.log(predictions+1e-15)+(1-y)*np.log(1-predictions+1e-15))
    print(f"Iteration {i+1}: Loss = {loss :.4f}")
    losses_gd.append(loss)

# Final weights
print("\nFinal weights:", weights)
```

```
Iteration 1: Loss = 0.6931
Iteration 2: Loss = 0.6507
Iteration 3: Loss = 0.6120
Iteration 4: Loss = 0.5766
Iteration 5: Loss = 0.5444
Iteration 6: Loss = 0.5150
Iteration 7: Loss = 0.4880
Iteration 8: Loss = 0.4633
Iteration 9: Loss = 0.4407
Iteration 10: Loss = 0.4198
```

```
Iteration 11: Loss = 0.4006
Iteration 12: Loss = 0.3829
Iteration 13: Loss = 0.3665
Iteration 14: Loss = 0.3513
Iteration 15: Loss = 0.3372
Iteration 16: Loss = 0.3241
Iteration 17: Loss = 0.3119
Iteration 18: Loss = 0.3005
Iteration 19: Loss = 0.2899
Iteration 20: Loss = 0.2799
```

```
Final weights: [1.30726898e-04 1.30726898e-04 6.54453300e-01 6.24226517e-01]
```

Method used to initialize weights: Zero initialization method

Reasoning: This approach of initializing weights is more suitable for logistic regression as it is a convex optimization problem which means that the loss function has a single global minimum. Starting with zeros ensures a simple, deterministic setup and all features are treated equally at initialization.

Loss Function used: Binary cross entropy loss

Reasoning: This is the most appropriate choice for binary classification problems such as logistic regression. This loss function measures the difference between the predicted probabilities and the actual binary labels by penalizing incorrect confident predictions more heavily.

4. Implementing Newton's method

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# Step 1: Generate and transform data
np.random.seed(0)
centers = [[-5, 0], [5, 1.5]]
X, y = make_blobs(n_samples=2000, centers=centers, random_state=5)
transformation = [[0.5, 0.5], [-0.5, 1.5]]
X = np.dot(X, transformation)

# Step 2: Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Step 3: Add bias term (intercept)
X = np.hstack((np.ones((X.shape[0], 1)), X)) # shape = (2000, 3)

# Step 4: Initialize weights
weights = np.zeros(X.shape[1]) # 3 weights (bias + 2 features)

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Newton's Method Training Loop
n_iter = 20
losses_newton=[]

for i in range(n_iter):
    z = np.dot(X, weights)
    predictions = sigmoid(z)

    # Gradient: shape = (3,)
    gradient = np.dot(X.T, predictions - y)

    # Hessian: shape = (3,3)
    S = predictions * (1 - predictions) # shape = (2000,)
    S_matrix = np.diag(S) # Convert to diagonal matrix
    H = np.dot(X.T, np.dot(S_matrix, X))

    # Update weights using Newton's update rule
    try:
        H_inv = np.linalg.inv(H) # Inverse of Hessian
        weights -= np.dot(H_inv, gradient)
    except np.linalg.LinAlgError:
        print(f"Iteration {i+1}: Hessian is singular. Stopping.")
```

```

break

# Compute and print loss
loss = -np.mean(y * np.log(predictions + 1e-15) + (1 - y) * np.log(1 - predictions + 1e-15))
print(f"Iteration {i+1}: Loss = {loss:.4f}")
losses_newton.append(loss)

# Final weights
print("\nFinal weights:", weights)

↩
Iteration 1: Loss = 0.6931
Iteration 2: Loss = 0.1452
Iteration 3: Loss = 0.0528
Iteration 4: Loss = 0.0203
Iteration 5: Loss = 0.0080
Iteration 6: Loss = 0.0032
Iteration 7: Loss = 0.0013
Iteration 8: Loss = 0.0005
Iteration 9: Loss = 0.0002
Iteration 10: Loss = 0.0001
Iteration 11: Loss = 0.0000
Iteration 12: Loss = 0.0000
Iteration 13: Loss = 0.0000
Iteration 14: Loss = 0.0000
Iteration 15: Loss = 0.0000
Iteration 16: Loss = 0.0000
Iteration 17: Loss = 0.0000
Iteration 18: Loss = 0.0000
Iteration 19: Loss = 0.0000
Iteration 20: Loss = 0.0000

Final weights: [-2.00629984 23.57768811 17.20102256]

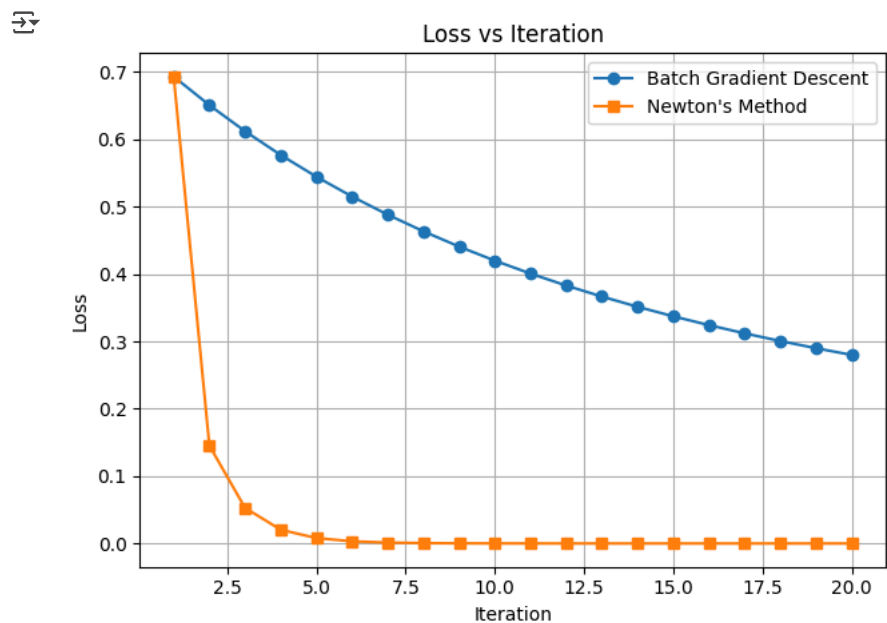
```

5. Loss curves for batch gradient descent and newton's method

```

plt.plot(range(1, len(losses_gd) + 1), losses_gd, label='Batch Gradient Descent', marker='o')
plt.plot(range(1, len(losses_newton) + 1), losses_newton, label="Newton's Method", marker='s')
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Loss vs Iteration")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



According to the above plot batch gradient descent shows a gradual and steady decrease in loss over 20 iterations indicating a slow but consistent improvement. This is because it only uses gradient information to update the weights and requires many small steps to reach the optimal situation.

On the other hand, Newton's method shows a rapid decrease in loss just within few iterations. This is because second order optimization uses both the gradient and curvature (via the Hessian matrix) to make large, accurate updates that lead to faster convergence.

6. Approaches to decide number of iterations for Gradient descent and Newton's method:

1. Use a **convergence threshold** on loss change - In this method, the algorithm continues updating the weights until the absolute difference between the current and previous loss values falls below a small predefined threshold (e.g., $\epsilon = 10$). This indicates that the model has reached a point where further updates result in negligible improvement, meaning it has likely converged to or near the optimal solution.
 2. Set a maximum number of iterations with early stopping - **Early stopping** can be applied by checking whether there is no significant improvement in a performance metric such as validation loss or gradient norm for a set number of consecutive iterations. This ensures that the training halts when learning becomes stagnant, even if the maximum iteration count has not been reached.
-

7. Convergence behavior after changing centers

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

# Step 1: New data with updated centers
np.random.seed(0)
centers = [[2, 2], [5, 1.5]]
X, y = make_blobs(n_samples=2000, centers=centers, random_state=5)
transformation = [[0.5, 0.5], [-0.5, 1.5]]
X = np.dot(X, transformation)

# Step 2: Feature scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Step 3: Add bias term
X = np.hstack((np.ones((X.shape[0], 1)), X)) # shape: (2000, 3)

# Step 4: Initialize weights
weights = np.zeros(X.shape[1]) # 3 weights (including bias)

# Step 5: Define sigmoid and loss
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

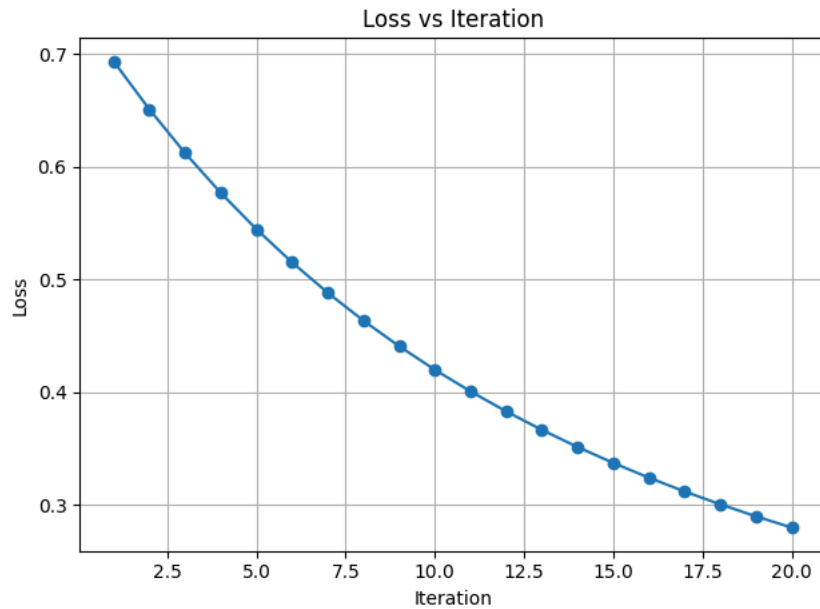
def compute_loss(y, predictions):
    return -np.mean(y * np.log(predictions + 1e-15) + (1 - y) * np.log(1 - predictions + 1e-15))

# Step 6: Batch gradient descent
alpha = 0.1
n_iter = 20
losses = []

for i in range(n_iter):
    z = np.dot(X, weights)
    predictions = sigmoid(z)
    gradient = np.dot(X.T, (predictions - y)) / X.shape[0]
    weights -= alpha * gradient
    loss = compute_loss(y, predictions)
    losses.append(loss)
    print(f"Iteration {i+1}: Loss = {loss:.4f}")

plt.plot(range(1, len(losses_gd) + 1), losses_gd, label='Batch Gradient Descent', marker='o')
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Loss vs Iteration")
plt.grid(True)
plt.tight_layout()
plt.show()
```

↺ Iteration 1: Loss = 0.6931
Iteration 2: Loss = 0.6769
Iteration 3: Loss = 0.6615
Iteration 4: Loss = 0.6467
Iteration 5: Loss = 0.6327
Iteration 6: Loss = 0.6193
Iteration 7: Loss = 0.6066
Iteration 8: Loss = 0.5944
Iteration 9: Loss = 0.5828
Iteration 10: Loss = 0.5717
Iteration 11: Loss = 0.5611
Iteration 12: Loss = 0.5510
Iteration 13: Loss = 0.5413
Iteration 14: Loss = 0.5321
Iteration 15: Loss = 0.5232
Iteration 16: Loss = 0.5147
Iteration 17: Loss = 0.5066
Iteration 18: Loss = 0.4988
Iteration 19: Loss = 0.4914
Iteration 20: Loss = 0.4842



In the new configuration the 2 clusters are closer together in feature space therefore the resulting classes after applying linear transformation and standardization are less easily separable.

Therefore the convergence is slower and the final loss is higher than the previous scenario due to overlapping distributions. Although the batch gradient descent still converges the gradient magnitudes are smaller and the decision boundary has a less confident margin between classes.